Python-Basics

Objective

- Introduction to Python
- Installation of Python
- Keywords
- Identifier
- Variables
- Data Types
- Operators
- Input/output
- Your First Python Program
- Comments

Introduction to Python

OBJECT™ TECHNOLOGIES

- Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.
- Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.
- **Python used in**

• **Data Science** • **Date Mining** • **Desktop Applications** • **Console-based Applications** • **Mobile Applications** • **Artificial Intelligence**	• **Web Applications** • **Enterprise Applications** • **3D CAD Applications** • **Machine Learning** • **Computer Vision or Image Processing Applications.** • **Speech Recognitions**

.co.in

Features of Python

OBJECT™ TECHNOLOGIES

- High-level Languages
- Open Source
- Portable
- Easy to use and Learn
- Object-Oriented Language
- GUI Programming Support
- Interpreted Language
- Wide Range of Libraries and Frameworks

Diff Between Interpreter and Compiler

OBJECT™ TECHNOLOGIES

Interpreter	Compiler
Translates program one statement at a time.	Scans the entire program and translates it as a whole into machine code.
Interpreters usually take less amount of time to analyze the source code. However, the overall execution time is comparatively slower than compilers.	Compilers usually take a large amount of time to analyze the source code. However, the overall execution time is comparatively faster than interpreters.
No Object Code is generated, hence are memory efficient.	Generates Object Code which further requires linking, hence requires more memory.
Interpreter converts code into byte code Hence Code is portable.	Code is not portable.
Programming languages like JavaScript, Python, Ruby use interpreters.	Programming languages like C, C++, Java use compilers.

Python Libraries

- Numpy
- Pandas
- Keras
- PyTorch
- LightGBM
- Eli5
- SciPy
- Theano
- TensorFlow
- Scikit-Learn

Installation of Python

OBJECT™ TECHNOLOGIES

Step 1:

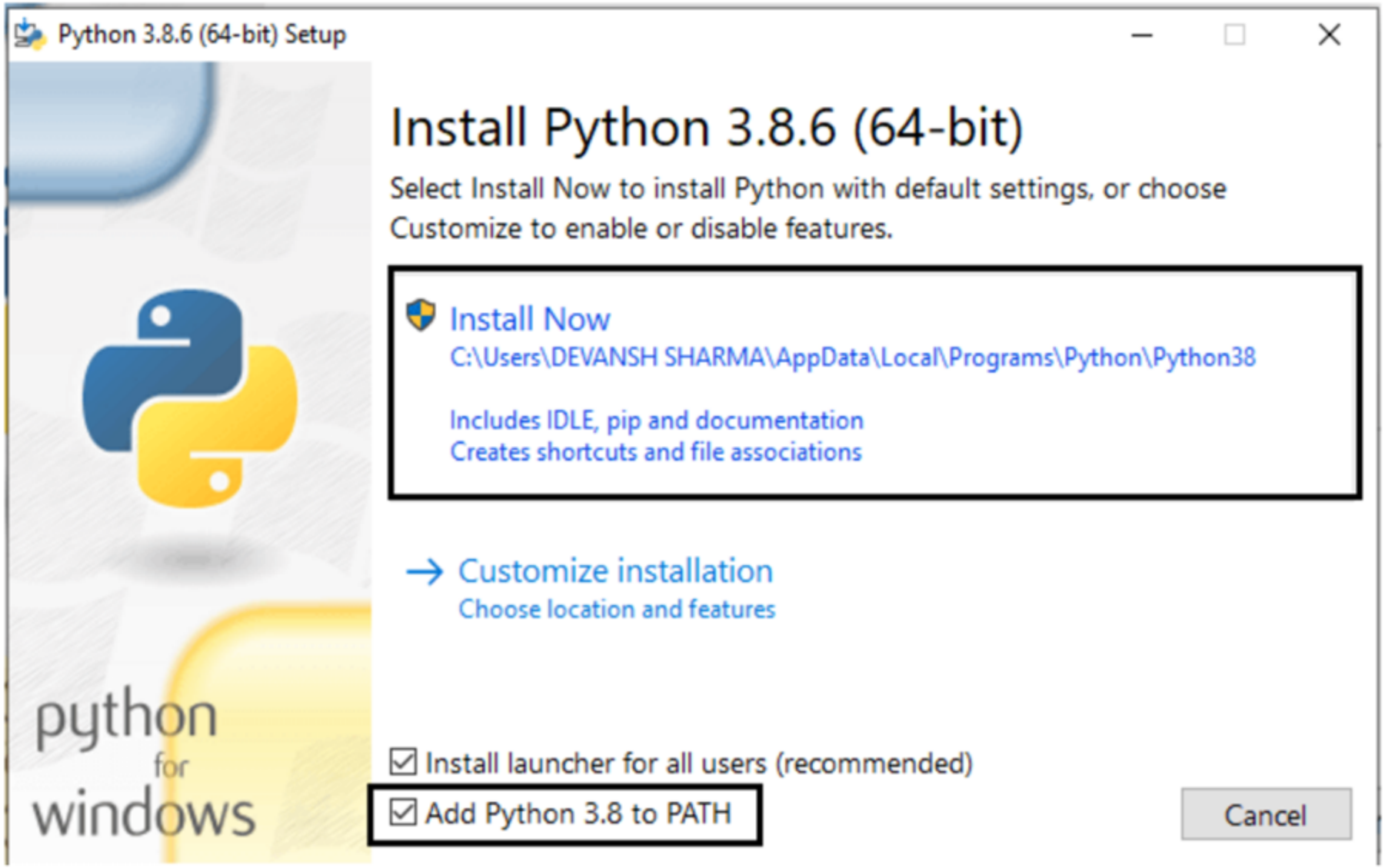
Many PCs and Macs will have python already installed. So first you have to check if you have python installed on your machine.

Go to cmd prompt -> python --version

Step 2:

If you find that you do not have python installed on your computer, then you can download it for free from the following website:

https://www.python.org/


Python 3.8.6 (64-bit) Setup
Install Python 3.8.6 (64-bit)
Select Install Now to install Python with default settings, or choose Customize to enable or disable features.
Install Now
C:\Users\DEVANSH SHARMA\AppData\Local\Programs\Python\Python38
Includes IDLE, pip and documentation
Creates shortcuts and file associations
Customize installation
Choose location and features
python
for
windows
Install launcher for all users (recommended)
Add Python 3.8 to PATH
Cancel

Python Versions and IDE

OBJECT™ TECHNOLOGIES

Python Versions

1. Python 2.0
2. Python 3.0
3. Latest version is 3.9. 0

IDE

- PyCharm
- kite
- Spyder
- IDLE
- Sublime
- Visual Studio Code
- Jupyter
- Pydev

Keywords

OBJECT™ TECHNOLOGIES

- Python Keywords are special reserved words that convey a special meaning to the compiler/interpreter. Each keyword has a special meaning and a specific operation.
  These keywords **can't** be used as a variable.
- **Following are keywords in python.**

False	None	True	and	as
assert	async	await	break	class
continue	def	elif	del	else
except	finally	for	from	global
if	import	in	lambda	nonlocal
is	not	pass	raise	or
return	try	with	yield	while

www.object.co.in

Identifier

OBJECT™ TECHNOLOGIES

An identifier is a name given to entities like class, functions, variables, etc. It helps to differentiate one entity from another.

Rules for writing identifiers:

- Identifiers can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore _.
- An identifier cannot start with a digit.
- Keywords cannot be used as identifiers.
- We cannot use special symbols like !, @, #, $, % etc. in our identifier.only _ is allowed
- spaces are not allowed. An identifier can be of any length.
- Identifier names are case sensitive.

Variables

- Variables are containers/reserved memory location for storing values.
- Unlike other programming languages, Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.
- Variables do not need to be declared with any particular type and can even change type after they have been set.
- String variables can be declared either by using single or double quotes.
- variable names are case-sensitive and they follow identifier rules.

Example:

```
x = 5
y = "John"
z= 1.26
```


www.object.co.in

Multiple Assignment

- Python allows us to assign a value to multiple variables in a single statement, which is also known as multiple assignments.

1. **Assigning single value to multiple variables**

```
x=y=z=50
print(x)
print(y)
print(z)
```

Output:

```
50
50
50
```


www.object.co.in

2. **Assigning multiple values to multiple variables**:

```
a,b,c=5,10,15
print (a)
print (b)
print (c)
```

Output:

```
5
10
15
```

Data Types

Python has the following data types:

- Text Type: str
- Numeric Types: int, float, complex
- Sequence Types: list, tuple
- Mapping Type: dict
- Set Types: set, frozenset
- Boolean Type: bool
- Binary Types: bytes, bytearray, memoryview

www.object.co.in

Example of Data Type

Text Type: str	**Sequence Types: tuple**
name = "Radha Deshmukh"	Subjects = ("c", "java", "python", "ML")
Numeric Types: int	**Mapping Type: dictionary**
prn = 210543083026	result = {210543083026:84, 210543083007:96}
Numeric Types: float	**Set Types: set**
marks = 80.5	AbsentPrn = {13, 34, 4}
Numeric Types: complex	**Boolean Type: bool**
x =12+2j	flag = True/False
Sequence Types: list	**Binary Types:bytes, bytearray, memoryview**
lst = [”Ovi Pawar", "Ram Mane", "Nakul Dev"]	Image= bytes(100)

Operators

- **Arithmetic operators** (+ (addition), - (subtraction), *(multiplication), /(divide) ,%(reminder), //(floor division), and exponent (**))
- **Comparison operators** (==, != , <, >,<=,>= etc.)
- **Assignment Operators** (+=, - = , *=, /= , //=, %= , **=etc.)
- **Logical Operators** (and , or, not)
- **Bitwise Operators** (&, | , ^ , ~ , << , >>)
- **Membership Operators** (in , not in)
- **Identity Operators** (is, is not)

False Values:

None
False
zero of any numeric type, for example, 0, 0.0, 0j.
any empty sequence, for example, '', (), [].
any empty mapping, for example, {}.

www.object.co.in

- You can use logical and condition as:

```
y=10
y>5 and y<20
True
```

Same code can be written as:

```
5<y<20
True
```

Membership Operators

- Python membership operators are used to check the membership of value inside a Python data structure. If the value is present in the data structure, then the resulting value is true otherwise it returns false.
- **in** - It is evaluated to be true if the first operand is found in the second operand (list, tuple, or dictionary). e.g x in y
- **not in** - It is evaluated to be true if the first operand is not found in the second operand (list, tuple, or dictionary). e.g x not in y

```
>>> x=5
>>> x in [10,3,5,6]
True
>>> x in [10,3,6]
False
>>> x not in [10,3,6]
True
```


www.object.co.in

Identity Operators

- The identity operators are used to decide whether an element certain class or type.
- **is** - Returns True if both variables are the same object.

  e.g x is y
- **is not** - Returns True if both variables are not the same object.

  e.g x is not y

```
>>> x=5
>>> y=10
>>> x is y
False
>>> x is not y
True
```


www.object.co.in

Python Operators Precedence

1. ** The exponent operator is given priority over all the others used in the expression.
2. ~ ,+ ,- The negation, unary plus, and minus.
3. * ,/, %, // The multiplication, divide, modules, reminder, and floor division.
4. + ,- Binary plus, and minus
5. >> ,<< Left shift. and right shift
6. & Binary and.
7. ^ ,| , Binary xor, and or
8. <= ,< ,> ,>= Comparison operators (less than, less than equal to, greater than, greater then equal to).
9. <>, == ,!= Equality operators.
10. = ,%= ,/=, //=, -=
11. += ,*= ,**= , Assignment operators
12. is ,is not Identity operators
13. in ,not in Membership operators
14. not ,or ,and Logical operators

www.object.co.in

Input/output

OBJECT TECHNOLOGIES™

Python Output Using print() function

We use the print() function to output data to the standard output device (screen).

```
print('This sentence is output to the screen')
a = 5
print('The value of a is', a)
print(1, 2, 3, 4)
print(1, 2, 3, 4, sep='*')
print(1, 2, 3, 4, sep='#', end='&')
```

Output

```
This sentence is output to the screen
The value of a is 5
1 2 3 4
1*2*3*4
1#2#3#4&
```


www.objectpro.in

Python Input

In Python, we have the input() function to get value from user.

The syntax for input() is:
input([prompt])
where prompt is the string we wish to display on the screen. It is optional.

```
>>> num = input('Enter a number: ')
   Enter a number: 10
>>> num
   here output is '10'
```

Here, we can see that the entered value 10 is a string, not a number. To convert this into a number we can use int() or float() functions.

```
>>> int('10')
10
>>> float('10')
10.0
```


www.object.co.in

First Python Program

OBJECT™ TECHNOLOGIES

Python provides us the two ways to run a program:

1. Using Interactive interpreter prompt
2. Using a script file

1. Interactive interpreter prompt

Python provides us the feature to execute the Python statement one by one at the interactive prompt.

To open the interactive mode, open the terminal (or command prompt) and type python

2.Using a script file

We need to write our code into a file which can be executed later. For this purpose, open an editor like notepad, create a file named first.py (Python used .py extension) and write the following code in it.

```
print ("hello world");
```

open cmd prompt type - python first.py

www.object.co.in

Comments

OBJECT™ TECHNOLOGIES

Single-Line Comments in Python

hash symbol # to write a single-line comment.

Example 1:

```
# printing a string
print('Hello world')
```

Output

Hello world

Multi-Line Comments in Python

Python doesn't offer a separate way to write multiline comments. However, there are other ways to get around this issue.

1. using # on each line
2. write comment in ('‘’)
3. write comment in (""")

www.object.co.in

Summary

- Introduction to Python
- Installation of Python
- Keywords
- Identifier
- Variables
- Data Types
- Operators
- Input/output
- Your First Python Program
- Comments

Thank You